

CSE312
SOFTWARE ARCHITECTURE PRINCIPLES AND PRACTICES

HOSTEL BUDDY SOFTWARE TEST REPORT

GROUP 19

Project Title

Hostel Buddy Smart Hostel Complaint Management System

Team Members and Roll Numbers

Team Member	Roll Number
Pranjal Tyagi	2024BCD0053
Dinesh Saini	2024BCD0033
Ajinkya	2024BCS0173
Vinay	2024BCS0249
Saurabh	2024BCS0185

Test Report Version 1.0

Date 22 September 2026

Repository revision 873307fcbdddc6fcbe5acab722ea0a2a810b7f48

1. Introduction

This report records the testing completed for Hostel Buddy, a web application for managing hostel maintenance complaints. The evidence shows that the current revision passed all 314 checks in the project test runner and all 12 report-specific unit checks. The report also records measured performance, security and reliability results, identified defects, and regression outcomes.

1.1 Purpose of Testing

Testing verifies that students, hostel managers, and the super admin can use the implemented workflows correctly and that role boundaries are enforced. It validates complaint creation and ownership, hostel-scoped administration, attachment handling, dashboards, the forward-only complaint lifecycle, smart priority and SLA assessment, hotspot analytics, and duplicate detection.

1.2 Scope of Testing

In Scope	Out of Scope
Student registration, login, profile, and role guards	Email or push notifications
Hostel and manager administration	Cloud deployment and production infrastructure
Complaint CRUD, lifecycle, remarks, search, filters, and pagination	External object storage and CDN behavior
Image and video upload validation and cleanup	Native mobile applications
Student, manager, and super-admin dashboards	Long-duration soak and internet-scale load testing
Priority/SLA, hotspot analytics, and duplicate detection	Formal accessibility conformance certification

2. System Under Test

2.1 System Description

Hostel Buddy replaces a paper complaint register with a browser-based workflow. Students register against a hostel, submit maintenance complaints with optional media, and track progress. Managers work only with their assigned hostel, while the super admin manages all hostels, managers, complaints, students, and institution-wide analytics.

2.2 Major Modules and Components

Module or Component	Implemented Responsibility
Authentication	Registration, login, JWT creation, password hashing, and role guards
Users	Profiles, student lists, manager provisioning, and hostel relationships
Hostels	Public hostel lookup and super-admin create, update, and delete operations
Complaints	Creation, media, ownership, editing, lifecycle, search, filters, and staff actions
Smart Triage and SLA	Explainable priority score, deadline policy, and live SLA state
Duplicate Detection	Thirty-day active-complaint comparison and persisted match links
Dashboard and Hotspots	Role-scoped statistics, trends, activity, and room-level risk ranking
Persistence	SQLite schema, repositories, migration helpers, seeding, and transactions
Frontend	Responsive HTML, CSS, vanilla JavaScript, forms, tables, dialogs, and charts

2.3 Technology Stack

Component	Technology or Tool
Frontend	HTML5, CSS3, vanilla JavaScript, vendored Chart.js
Backend	Node.js v24.18.0 and Express 4.21.2

Component	Technology or Tool
Database	Node built-in node:sqlite with SQLite WAL mode
Authentication	JSON Web Token and bcryptjs
Security	Helmet, Content Security Policy, role middleware, auth rate limiting
Uploads	Multer with byte-signature validation
Automation	Node.js scripts, Fetch API, assert module, npm test

3. Test Strategy

Testing combines direct unit checks of deterministic functions with black-box HTTP tests against a running application and isolated SQLite database. The main runner seeds known data, starts the server on port 4010, executes six suites, and removes the test database and uploads. Manual browser verification supplements the automated API coverage for desktop and mobile presentation.

3.1 Testing Levels

Testing Level	Performed	Modules or Components Tested
Unit Testing	Yes	Triage, duplicate similarity, and validation helpers
Integration Testing	Yes	Routes, middleware, services, repositories, SQLite, uploads, and dashboards
System Testing	Yes	Complete complaint and administration workflows through the API and browser
Validation Testing	Yes	Implemented requirements mapped to executed checks
Regression Testing	Yes	Forty-nine automated checks for previously corrected defects and smart triage

3.2 Test Environment

Item	Actual Environment
Operating System	Windows win32 10.0.26200
Processor	16 logical CPUs reported by Node.js os.cpus()
Browser	Chromium-based browser used for UI walkthrough; exact version not recorded
Programming Language	JavaScript on Node.js v24.18.0
Framework	Express 4.21.2
Database	Isolated SQLite test databases under data/
Test Framework	Project Node.js check harness plus Node assert for unit tests
Automation Command	npm test
Repository Revision	873307fcbdddc6fcbe5acab722ea0a2a810b7f48

3.3 Pass and Fail Criteria

A functional test passes when the observed HTTP status, response data, database effect, file effect, authorization outcome, or user-interface behavior equals the stated expected result. A suite fails if any check fails or if the isolated server cannot start. Performance tests pass only when every request succeeds and the measured 95th percentile stays below the implemented NFR threshold.

4. Test Case Design

Test cases use equivalence classes, boundary values, state transitions, authorization partitions, negative input, interface checks, and realistic workflows. The following cases are representative; Section 11 lists the complete automated suites.

Test Case ID	Module	Scenario	Precondition	Input	Expected Result	Actual Result	Status
TC-AUTH-01	Auth	Valid student login	Seeded account	Correct email and password	200 and JWT	200 and JWT returned	Pass
TC-AUTH-02	Auth	Wrong password	Existing account	Wrong password	Generic 401	Generic 401 returned	Pass
TC-COMP-01	Complaints	Create complaint	Student logged in	Valid category and description	201 and Pending	201 and Pending	Pass
TC-COMP-02	Complaints	Edit non-Pending complaint	Complaint In Progress	PUT request	409 NOT_PENDING	409 NOT_PENDING	Pass
TC-OWN-01	Access	Read another student's complaint	Two students	Student B reads A	403	403	Pass
TC-SCOPE-01	Manager	Read another hostel complaint	Two managers	Manager A reads B	403	403	Pass
TC-LIFE-01	Lifecycle	Resolved to Pending	Resolved complaint	Backward transition	409 INVALID_TRANSITION	409 INVALID_TRANSITION	Pass
TC-FILE-01	Uploads	Spoofed PNG	Student logged in	Text bytes with image MIME	400 INVALID_FILE_TYPE	400 INVALID_FILE_TYPE	Pass
TC-TRIAGE-01	Triage	Safety hazard	Student logged in	Electricity plus sparking	Critical and 2 hours	Critical and 2 hours	Pass
TC-DUP-01	Duplicates	Likely repeat	Active related complaint	Matching text/category/room	Privacy-safe warning	Warning and score returned	Pass
TC-HOT-01	Hotspots	Room cluster	Three room complaints	GET 30-day hotspots	Ranked actionable cluster	Ranked actionable cluster	Pass
TC-PAGE-01	Pagination	Page past end	Multiple complaints	page=99999	Clamp to final page	Final page returned	Pass

5. Black-Box Testing

Black-box checks call public HTTP endpoints without relying on internal implementation details. Equivalence classes cover valid and invalid credentials, roles, categories, status values, files, periods, and pagination. Boundary checks cover password bytes, upload size, and allowed hotspot periods.

Test ID	Module	Technique	Input or Condition	Expected Output	Actual Output	Result
BB-01	Registration	Equivalence class	Valid complete student payload	201 and token	201 and token	Pass
BB-02	Registration	Invalid class	Unknown hostel	400	400	Pass
BB-03	Authentication	Invalid class	Wrong password	Generic 401	Generic 401	Pass
BB-04	Password	Boundary value	72 ASCII characters	Accepted	201	Pass
BB-05	Password	Boundary value	80 ASCII characters	Rejected	400	Pass
BB-06	Hotspots	Boundary value	days=7	Accepted	200	Pass
BB-07	Hotspots	Invalid class	days=14	Rejected	400	Pass
BB-08	Uploads	Negative test	Text bytes declared image/png	Rejected	400	Pass
BB-09	Search	Special input	Literal percent and underscore	Not wildcard-all	Correct literal behavior	Pass
BB-10	Duplicates	Privacy partition	Student preflight	No identity or description leak	Private fields absent	Pass

6. White-Box Testing

White-box tests directly import selected deterministic modules and exercise important branches. The report unit harness covers triage thresholds and signal handling, duplicate acceptance and rejection paths, result limits, type validation, integer conversion, byte-length handling, and multipart truth flags.

Test ID	Module or Function	Technique	Paths or Conditions Tested	Result
---------	--------------------	-----------	----------------------------	--------

Test ID	Module or Function	Technique	Paths or Conditions Tested	Result
WB-01	assessComplaint	Branch test	Furniture baseline -> Low -> 72 hours	Pass
WB-02	assessComplaint	Branch test	Electricity plus sparking -> cap 100 -> Critical -> 2 hours	Pass
WB-03	assessComplaint	Determinism	Identical input produces identical object	Pass
WB-04	significantTokens	Normalization	Wi-Fi, wi fi, and WIFI collapse to wifi	Pass
WB-05	assessPair	True branch	Exact text, category, and room -> score 100	Pass
WB-06	assessPair	False branch	Unrelated text rejected despite contextual boosts	Pass
WB-07	findDuplicateMatches	Limit branch	Four matches reduced to strongest three	Pass
WB-08	toPositiveInt	Decision paths	Valid integer, partial text, and zero	Pass
WB-09	isTruthyFlag	Decision paths	on and yes accepted; maybe rejected	Pass

6.1 Coverage

Module or Function	Statement Coverage	Branch Coverage	Evidence
triage.js	N/A	N/A	WB-01 to WB-03 and UNIT-E01
duplicateDetection.js	N/A	N/A	WB-04 to WB-07 and UNIT-E01
validators.js	N/A	N/A	WB-08 to WB-09 and UNIT-E01

Statement and branch percentages are reported as N/A because the repository does not contain an instrumented coverage tool or generated coverage report. The executed path evidence above is factual, but it is not presented as numerical code coverage.

7. Unit Testing

A report-specific Node.js unit harness imported three dependency-free production modules. It ran without the server or database and produced 12 passes and 0 failures.

Test ID	Module or Function	Test Input	Expected Result	Actual Result	Status
UT-01	assessComplaint	Furniture, worn chair	Low, score 20, SLA 72	Low, score 20, SLA 72	Pass
UT-02	assessComplaint	Electricity, sparking wire	Critical, score 100, SLA 2	Critical, score 100, SLA 2	Pass
UT-03	assessComplaint	Same object twice	Deep equal	Deep equal	Pass
UT-04	significantTokens	Wi-Fi wi fi WIFI	[wifi]	[wifi]	Pass
UT-05	assessPair	Exact text/category/room	Similarity 100	Similarity 100	Pass
UT-06	assessPair	Unrelated text, same context	null	null	Pass
UT-07	findDuplicateMatches	Four matching candidates	Three results	Three results	Pass
UT-08	isNonEmptyString	Object input	false	false	Pass
UT-09	isEmail	Valid and invalid syntax	true and false	true and false	Pass
UT-10	byteLength	ASCII a and Devanagari na	1 and 3 bytes	1 and 3 bytes	Pass
UT-11	toPositiveInt	25, 25abc, 0	25, null, null	25, null, null	Pass
UT-12	isTruthyFlag	on, yes, maybe	true, true, false	true, true, false	Pass

7.1 Unit Test Evidence

Figure UNIT-E01 Actual execution output from the report unit harness

Report unit harness - core deterministic functions

```
PASS UT-01 Furniture baseline produces Low priority and 72-hour SLA
PASS UT-02 Electricity plus sparking is capped at Critical score 100
PASS UT-03 Triage is deterministic for identical input
PASS UT-04 Wi-Fi spelling variants normalize to one token
PASS UT-05 Exact same-category same-room duplicate scores 100
PASS UT-06 Unrelated descriptions do not match despite contextual boosts
PASS UT-07 Duplicate results are limited to the three strongest matches
PASS UT-08 Object input is not accepted as a non-empty string
PASS UT-09 Email validation accepts valid and rejects invalid syntax
PASS UT-10 Password byte length counts multibyte UTF-8 input
PASS UT-11 Positive integer conversion rejects partial and non-positive input
PASS UT-12 Multipart truth flags reject arbitrary text
UNIT RESULT 12 passed 0 failed
```

8. Integration and Interface Testing

Integration tests start the complete Express application against a seeded throwaway database. They verify route, middleware, service, repository, database, filesystem, and frontend API contracts through HTTP.

Test ID	Component 1	Component 2	Interface	Integration Scenario	Expected Result	Actual Result	Status
INT-01	Auth route	User repository	HTTP and SQL	Register student	User and subtype stored	Stored and token returned	Pass
INT-02	Complaint service	Triage	Function call	Create complaint	Triage row stored	Priority and SLA returned	Pass
INT-03	Complaint service	Duplicate engine	Function call	Submit likely repeat	Match linked	Link persisted	Pass
INT-04	Upload middleware	Filesystem	Multipart	Replace video	Old file removed	Old URL returned 404	Pass
INT-05	Manager auth	Complaint repo	Scoped SQL	List own hostel	No foreign rows	Only own hostel rows	Pass
INT-06	Dashboard service	Complaint repo	SQL aggregate	Load staff dashboard	Consistent totals	All sums consistent	Pass
INT-07	Hotspot route	Analytics repo	HTTP and SQL	Request 30-day ranking	Ranked actions	Ranked actions returned	Pass

Test ID	Component 1	Component 2	Interface	Integration Scenario	Expected Result	Actual Result	Status
INT-08	Status service	Database	PATCH and SQL	Resolve complaint	resolved_at stored once	Timestamp stable	Pass

8.1 Integration Test Evidence

Evidence AUTO-E01 records the full six-suite run. Script hashes and the repository revision in Appendix A identify the exact executed sources.

8.2 Integration Approach

The project used incremental integration. Routes were connected to middleware, controllers, services, repositories, and SQLite in module-sized increments. The final runner executes the integrated application in a fresh environment, which prevents one suite from depending on another suite's database state.

9. System Testing

System tests exercise realistic workflows across multiple modules. The automated flows use public APIs; a browser walkthrough verified that the corresponding interfaces render and behave on desktop and mobile layouts.

Test ID	End-to-End Scenario	Expected Result	Actual Result	Status
SYS-01	List hostels -> register -> login -> view and update profile	Authenticated student profile updated	All steps passed	Pass
SYS-02	Student creates complaint -> staff advances status -> student reads update	Status, remarks, and timestamps visible	All steps passed	Pass
SYS-03	Submit media -> serve media -> replace/remove -> delete complaint	No orphaned files	Old files returned 404	Pass
SYS-04	Manager lists, reads, and updates own hostel while denied another	Hostel boundary enforced	Own allowed; foreign denied	Pass
SYS-05	Duplicate preflight -> submit anyway -> persisted staff reference	Privacy-safe link	Warning and link passed	Pass
SYS-06	Create room cluster -> open dashboard -> change hotspot period	Rank, trend, scope, and action correct	All analytics checks passed	Pass

10. Validation Testing and Requirements Traceability

The table maps implemented requirements to executed evidence. It covers the current multi-hostel application and the three completed smart upgrades. The percentage in Section 16 applies only to this mapped set and is not presented as full SRS coverage.

Requirement ID	Implemented Requirement	Executed Test Evidence	Result
REQ-AUTH-01	Students register and all three roles log in	auth suite checks 1 and 3	Pass
REQ-AUTH-02	Duplicate email and roll number are rejected	auth suite check 2	Pass
REQ-PROFILE-01	Student views and updates own profile	auth suite checks 5 and 6	Pass
REQ-HOSTEL-01	Super admin manages hostels and managers	hostels suite checks 1 to 10	Pass
REQ-COMP-01	Student creates and reads own complaint	complaints suite checks 1, 6, and 9	Pass
REQ-COMP-02	Only Pending complaints can be edited or deleted	complaints suite checks 10 to 12	Pass
REQ-LIFE-01	Complaint lifecycle moves forward and records resolution	admin checks 10 to 14; regression BUG-02	Pass
REQ-ACCESS-01	Students cannot access another student's complaint	complaints suite check 8	Pass
REQ-SCOPE-01	Managers are confined to their assigned hostel	admin check 15 and auth check 8	Pass
REQ-DASH-01	Dashboards provide internally consistent statistics	dashboard checks 1 to 8	Pass
REQ-MEDIA-01	Images and videos are validated, served, and cleaned	complaints checks 2 to 5 and 13 to 19	Pass
UPG-01	Priority and SLA are generated and explainable	regression SMART TRIAGE and admin check 4b	Pass
UPG-02	Room hotspots are ranked, scoped, and actionable	dashboard check 5	Pass
UPG-03	Duplicates are warned, private, linked, and recalculated	complaints check 20	Pass

11. Automated Test Scripts and Execution

Automation is implemented as executable Node.js scripts. The main runner starts an isolated server and database, executes six suites, and cleans the test state. The unit and performance harnesses were executed specifically for this report.

11.1 Automated Test Script Summary

Script ID	Script or Test Case	Module	Tool	Automated Scope	Result
ATS-01	tests/auth.test.js	Auth and users	Node Fetch API	47 API and role checks	Pass
ATS-02	tests/hostels.test.js	Hostels and managers	Node Fetch API	46 CRUD and authorization checks	Pass
ATS-03	tests/complaints.test.js	Complaints	Node Fetch API	66 lifecycle, media, ownership, duplicate checks	Pass
ATS-04	tests/admin.test.js	Staff operations	Node Fetch API	58 search, filter, scope, status, dashboard checks	Pass
ATS-05	tests/dashboard.test.js	Dashboards	Node Fetch API	48 statistics, activity, hotspot, guard checks	Pass
ATS-06	tests/regression.test.js	Regression	Node Fetch API	49 corrected-defect and triage checks	Pass
ATS-07	unit-core.test.js	Core functions	Node assert	12 direct unit checks	Pass
ATS-08	performance-test.js	NFR	Node perf_hooks	230 measured HTTP requests	Pass

11.2 Selected Automated Test Script

Representative executed unit check from the report harness:

```
check('UT-02', 'Electricity plus sparking is capped at Critical score 100', () => {
  const result = assessComplaint({
    category: 'Electricity',
    description: 'Wire is sparking.'
  });
  assert.equal(result.priority, 'Critical');
  assert.equal(result.score, 100);
  assert.equal(result.slaHours, 2);
});
```

Representative orchestration behavior from tests/run.js:

```
removeTestDatabase();
removeTestUploads();
seedTestDatabase();
const server = startServer();
await waitForHealth(server);
for (const name of SUITES) await runSuite(name);
server.kill();
removeTestDatabase();
removeTestUploads();
```

11.3 Automated Test Execution Summary

Execution Type	Distinct Checks	Passed	Failed	Remarks
Report unit tests	12	12	0	Production functions imported directly
Automated API integration and regression	314	314	0	Six suites on isolated server and database
Performance requests	230	230	0	Measurements reported separately, not counted as functional checks
Unique functional checks	326	326	0	12 unit plus 314 project checks

11.4 Automated Test Evidence

Figure AUTO-E01 Actual npm test execution summary

npm test - isolated automated suite

```
> hostel-buddy@1.0.0 test
> node --disable-warning=ExperimentalWarning tests/run.js
[test] seeding the throwaway database (hostels, managers, students)
[test] starting server on port 4010 with a throwaway database
[test] server ready
auth.test.js      RESULT: 47 passed, 0 failed
hostels.test.js   RESULT: 46 passed, 0 failed
complaints.test.js RESULT: 66 passed, 0 failed
admin.test.js     RESULT: 58 passed, 0 failed
dashboard.test.js RESULT: 48 passed, 0 failed
regression.test.js RESULT: 49 passed, 0 failed
[test] all 6 suites passed - 314 checks passed, 0 failed
```

12. Non-Functional Testing

Four relevant quality attributes were tested. Performance and reliability used isolated automated runs. Security used executed negative and authorization checks. Usability used the completed desktop and 390 by 844 mobile browser walkthrough.

12.1 Non-Functional Test Scenarios

NFR ID	Quality Attribute	Test Scenario	Metric or Acceptance Criterion	Actual Result	Status
NFR-01	Performance	200 authenticated admin dashboard reads at concurrency 20	p95 below 3000 ms; zero failures	p95 25.22 ms; 200/200 successful	Pass
NFR-02	Performance	30 complaint submissions with triage and duplicate recheck	p95 below 2000 ms; zero failures	p95 20.18 ms; 30/30 successful	Pass
NFR-03	Security	Anonymous, wrong-role, cross-owner, cross-hostel, malformed input, spoofed file	Correct 400/401/403; no protected data or internals	All selected auth, admin, complaint, and regression checks passed	Pass
NFR-04	Reliability	Run all six suites twice and verify cleanup	Both runs 314/314; test DB/uploads absent after run	Run 1 and Run 2 passed; cleanup paths absent	Pass
NFR-05	Usability	Desktop and 390x844 browser walkthrough	No page-level overflow; controls usable; no console errors	Pages responsive; tables scroll internally; zero console errors	Pass

12.2 Non-Functional Testing Evidence

Evidence ID	NFR ID	Evidence Type	Description	Result
PERF-E01	NFR-01/02	Measured output	Node perf_hooks latency distribution and success counts	Pass
SEC-E01	NFR-03	Automated output	Auth, ownership, role, scope, validation, and upload checks in AUTO-E01	Pass
REL-E01	NFR-04	Repeated execution	Two complete runs plus filesystem cleanup checks	Pass
UI-E01	NFR-05	Browser observation	Desktop and mobile responsive walkthrough, console inspected	Pass

Figure PERF-E01 Actual measured local response-time output

Measured local performance - 22 September 2026
Environment: Node v24.18.0 Windows 10.0.26200 16 logical CPUs isolated SQLite database
Authenticated admin dashboard read
200 requests concurrency 20 200 successful 0 failed
average 21.03 ms p50 20.21 ms p95 25.22 ms p99 26.26 ms max 28.04 ms
throughput 890.58 requests/s criterion p95 < 3000 ms and zero failures PASS
Complaint submission with triage and duplicate recheck
30 requests 30 successful 0 failed
average 14.68 ms p50 15.65 ms p95 20.18 ms p99 21.05 ms max 21.05 ms
throughput 67.46 requests/s criterion p95 < 2000 ms and zero failures PASS

Figure REL-E01 Repeated suite execution and cleanup verification

Reliability repeat run and cleanup verification
Run 1: 6 suites 314 passed 0 failed
Run 2: 6 suites 314 passed 0 failed
Repeated executions: 628 checks 628 passed 0 failed
After Run 2: data/test.db exists = False
After Run 2: uploads/.test exists = False
Result: deterministic repeatability and isolated cleanup PASS

13. Defect and Bug Report

The project engineering log records defects found during implementation and a hostile-input audit. The table includes defects with direct regression evidence in the current suite. Severity is classified for this report according to security, data-integrity, and workflow impact.

Bug ID	Related Test	Module	Defect Description	Severity	Status	Resolution
BUG-01	REG-VAL	Validation	JSON objects were coerced to [object Object] and stored	High	Fixed	Type predicates now reject non-string input
BUG-02	REG-LIFE	Lifecycle	Resolved or Closed complaints could move backward	High	Fixed	Forward-only transition guard returns 409
BUG-03	REG-FILE	Uploads	Declared MIME type was trusted without checking bytes	High	Fixed	Magic-byte validation and deletion added
BUG-04	REG-ERROR	Error handling	Parser and Multer internals reached clients	Medium	Fixed	Stable user-facing error mapping added
BUG-05	REG-SEARCH	Search	Percent and underscore acted as LIKE wildcards	Medium	Fixed	LIKE metacharacters are escaped
BUG-06	REG-PASS	Authentication	bcrypt silently ignored password bytes after 72	Medium	Fixed	Passwords above 72 bytes are rejected
BUG-07	REG-PAGE	Pagination	Out-of-range page number was echoed back	Low	Fixed	Page is clamped to the final valid page
BUG-08	REG-ATTACH	Attachments	Students could replace but not remove media	Medium	Fixed	Explicit removal flags and cleanup added

14. Regression Testing

After each fix, the affected behavior and related workflows were re-executed. The dedicated regression suite passed 49 of 49 checks during both complete runs.

Test Case	Original Result	Defect Fixed	Retest Evidence	Retest Result
REG-VAL	Invalid object accepted	BUG-01	Five validation and persistence checks	Pass
REG-LIFE	Backward transition accepted	BUG-02	Eleven lifecycle consistency checks	Pass
REG-FILE	Spoofed PNG stored	BUG-03	Genuine PNG accepted; spoof rejected with code	Pass
REG-ERROR	Framework text exposed	BUG-04	Malformed JSON and unexpected field mapped safely	Pass
REG-SEARCH	Wildcard matched all	BUG-05	Percent, underscore, and normal name search checks	Pass
REG-PASS	Long password silently truncated	BUG-06	80 rejected; 72 accepted	Pass
REG-PAGE	page 99999 echoed	BUG-07	Page clamped and rows still returned	Pass
REG-ATTACH	Removal unavailable	BUG-08	URL cleared and old file returned 404	Pass
REG-FULL	Risk of collateral regression	All listed	Complete 314-check suite executed twice	Pass

15. Test Execution Summary

Counts below are mutually exclusive for the primary automated result. System, validation, black-box, and white-box classifications overlap these executed checks and are therefore not added again.

Testing Category	Distinct Checks	Passed	Failed	Pass Percentage
Unit Testing	12	12	0	100%
API Integration and Functional Testing	265	265	0	100%
Regression Testing	49	49	0	100%
Total Unique Functional Checks	326	326	0	100%
Reliability Repeat Run	314 repeated	314	0	100%
Performance Requests	230 requests	230	0	100%

16. Test Metrics and Coverage

Metric	Actual Value	Evidence and Interpretation
Unique functional checks	326 passed, 0 failed; 100%	12 unit plus 314 project checks; UNIT-E01 and AUTO-E01
Main suites	6 of 6 passed	auth, hostels, complaints, admin, dashboard, regression
Reliability repeat	628 of 628 executions passed	314 checks repeated twice; not unique
Mapped traceability coverage	14 of 14 mapped items passed	Applies only to Section 10 mapped set
Statement coverage	N/A	No instrumented coverage report exists
Branch coverage	N/A	No instrumented coverage report exists
Dashboard p95	25.22 ms	200 local authenticated reads at concurrency 20
Complaint submission p95	20.18 ms	30 local submissions with triage and duplicate recheck
Documented defects in this report	8 fixed	All eight have passing regression evidence

17. Test Evidence

Evidence ID	Description	Related Test, Bug, or NFR	Evidence Location
AUTO-E01	Actual npm test output showing six suites and 314 passes	ATS-01 to ATS-06	Section 11.4
UNIT-E01	Actual report unit harness output showing 12 passes	UT-01 to UT-12	Section 7.1
PERF-E01	Measured latency distribution and success counts	NFR-01 and NFR-02	Section 12.2
REL-E01	Two complete suite runs and cleanup verification	NFR-04	Section 12.2
SEC-E01	Authorization, ownership, scope, malformed-input, and upload checks	NFR-03	AUTO-E01 and suite output
UI-E01	Desktop and 390x844 browser walkthrough with console inspection	NFR-05 and SYS cases	Browser verification record
DEF-E01	Engineering defect log with root causes and fixes	BUG-01 to BUG-08	docs/problems_faced_and_bugs_encountered.md
SRC-E01	Script hashes and repository revision	All automation	Appendix A

Evidence was collected from repository revision 873307fcbdddc6fcb5acab722ea0a2a810b7f48 on 22 September 2026. The main and performance runners used isolated databases and upload directories; the development database was not used.

18. Known Limitations

- Statement and branch coverage percentages are unavailable because no coverage instrumentation is configured.
- Performance measurements were taken on one local Windows machine and do not represent production network latency or internet-scale load.
- The manual UI walkthrough used a Chromium-based browser only; Firefox and Safari were not tested in this report.
- Formal accessibility conformance, screen-reader behavior, and keyboard-only navigation were not audited.
- Authentication rate limiting is disabled under NODE_ENV=test by design, so the automated suite does not measure throttling behavior.
- Email, push notifications, cloud storage, and native mobile clients are outside the implemented scope.

19. Conclusion

Hostel Buddy passed all 326 distinct automated functional checks included in this report. The six project suites passed twice, and both measured performance scenarios completed with zero failed requests and p95 latency below the stated acceptance limits. Security-focused checks confirmed authentication, authorization, ownership, hostel scope, input validation, and file validation behavior. Eight documented defects were fixed and their regression checks passed.

The tested revision is suitable for the current academic demonstration scope. Remaining work is limited to the explicit items in Section 18, especially instrumented code coverage, broader browser and accessibility testing, and production-environment performance testing.

Appendix A Complete Automated Test Scripts

Complete project scripts are stored in the repository tests directory at revision 873307fcbdddc6fcb5e5acab722ea0a2a810b7f48. The SHA-256 values below identify the exact executed files. The report-specific unit and performance harness source is retained with the report working evidence.

Script	Checks	SHA-256
tests/auth.test.js	47	732FD119F0FC1AF4657A816130AF90F156C7303F41EC863B6CB0615635EAAEA2
tests/hostels.test.js	46	07A1ADE4AC2CA4B4CCEA1B128F83341FFD5F64DA46D7710ED5D938E12D2095D0
tests/complaints.test.js	66	A5A7C9CC12D7B14067701759B1B7723F801F88EBECC008819BBE97D34941B50
tests/admin.test.js	58	3D30F384A4D79D0EF846C7D6B9E1C80C7B0D2C3D033C1583AAEE29C98F73E4D5
tests/dashboard.test.js	48	789D918C7F7642EF70ADB77293376DCA4A5C57CFC4171997A024B59E3E7ECE87
tests/regression.test.js	49	AC9CC12BCD1E04A47712DC9715A88E35B9A0C3C989004BDA088CB19C10DC314B

Appendix A.1 Report Unit Harness

The unit harness uses Node assert and imports assessComplaint, significantTokens, assessPair, findDuplicateMatches, and shared validators directly. It contains the 12 checks listed in Section 7 and exits with a nonzero code if any assertion fails.

```
let passed = 0;
let failed = 0;
function check(id, description, fn) {
  try {
    fn();
    passed += 1;
    console.log(`PASS ${id} ${description}`);
  } catch (error) {
    failed += 1;
    console.log(`FAIL ${id} ${description}: ${error.message}`);
  }
}
console.log(`UNIT RESULT ${passed} passed ${failed} failed`);
process.exit(failed ? 1 : 0);
```

Appendix A.2 Execution Commands

```
npm test
node --disable-warning=ExperimentalWarning tmp\test-report\unit-core.test.js
node --disable-warning=ExperimentalWarning tmp\test-report\performance-test.js
```

Appendix B Additional Test Evidence

The following rendered excerpts reproduce actual console results collected during this report preparation. They are included at readable scale so the reported totals and measurements can be checked directly.

Figure B-01 Main automated suite output

```
npm test - isolated automated suite

> hostel-buddy@1.0.0 test
> node --disable-warning=ExperimentalWarning tests/run.js
[test] seeding the throwaway database (hostels, managers, students)
[test] starting server on port 4010 with a throwaway database
[test] server ready
auth.test.js      RESULT: 47 passed, 0 failed
hostels.test.js   RESULT: 46 passed, 0 failed
complaints.test.js RESULT: 66 passed, 0 failed
admin.test.js     RESULT: 58 passed, 0 failed
dashboard.test.js RESULT: 48 passed, 0 failed
regression.test.js RESULT: 49 passed, 0 failed
[test] all 6 suites passed - 314 checks passed, 0 failed
```

Figure B-02 Unit harness output

```
Report unit harness - core deterministic functions

PASS UT-01 Furniture baseline produces Low priority and 72-hour SLA
PASS UT-02 Electricity plus sparking is capped at Critical score 100
PASS UT-03 Triage is deterministic for identical input
PASS UT-04 Wi-Fi spelling variants normalize to one token
PASS UT-05 Exact same-category same-room duplicate scores 100
PASS UT-06 Unrelated descriptions do not match despite contextual boosts
PASS UT-07 Duplicate results are limited to the three strongest matches
PASS UT-08 Object input is not accepted as a non-empty string
PASS UT-09 Email validation accepts valid and rejects invalid syntax
PASS UT-10 Password byte length counts multibyte UTF-8 input
PASS UT-11 Positive integer conversion rejects partial and non-positive input
PASS UT-12 Multipart truth flags reject arbitrary text
UNIT RESULT 12 passed 0 failed
```

Figure B-03 Performance measurement output

Measured local performance - 22 September 2026

```
Environment: Node v24.18.0 | Windows 10.0.26200 | 16 logical CPUs | isolated SQLite
database
Authenticated admin dashboard read
200 requests | concurrency 20 | 200 successful | 0 failed
average 21.03 ms | p50 20.21 ms | p95 25.22 ms | p99 26.26 ms | max 28.04 ms
throughput 890.58 requests/s | criterion p95 < 3000 ms and zero failures | PASS
Complaint submission with triage and duplicate recheck
30 requests | 30 successful | 0 failed
average 14.68 ms | p50 15.65 ms | p95 20.18 ms | p99 21.05 ms | max 21.05 ms
throughput 67.46 requests/s | criterion p95 < 2000 ms and zero failures | PASS
```

Figure B-04 Repeatability and cleanup output

Reliability repeat run and cleanup verification

```
Run 1: 6 suites | 314 passed | 0 failed
Run 2: 6 suites | 314 passed | 0 failed
Repeated executions: 628 checks | 628 passed | 0 failed
After Run 2: data/test.db exists = False
After Run 2: uploads/.test exists = False
Result: deterministic repeatability and isolated cleanup PASS
```